Prueba técnica
Post-OCR Invoice Resolution Service

Documento de enunciado para candidato

Implementa un servicio backend que reciba facturas en formato post-OCR, las normalice, las almacene y utilice un agente basado en LLM para resolver campos de negocio simplificados apoyándose en histórico y datos de referencia. La solución debe exponer estos resultados por API y exportarlos incrementalmente a una segunda capa analítica.

Introducción

Implementa un servicio backend que reciba facturas en formato post-OCR, las normalice, las almacene y utilice un agente basado en LLM para resolver campos de negocio simplificados apoyándose en histórico y datos de referencia. La solución debe exponer estos resultados por API y exportarlos incrementalmente a una segunda capa analítica.

Objetivo

Implementar un servicio backend capaz de recibir, persistir, enriquecer y resolver facturas a partir de un payload estructurado equivalente a la salida de un sistema OCR/parsing documental.

La solución debe combinar tres dimensiones:

- manejo y modelado de datos
- exposición de una API
- uso de un componente agentic basado en LLM

El sistema debe ser capaz de tomar datos ya extraídos de una factura, normalizarlos, consultar contexto histórico o datos de referencia, y devolver una resolución estructurada de negocio.

Además, debe incluir un mecanismo de exportación incremental hacia una segunda capa de datos orientada a analítica, reporting o preparación de datasets.

Contexto

Se asume que existe una etapa previa de OCR/parsing que ya ha procesado los documentos y ha extraído información relevante.

Por tanto, la entrada del sistema no será un PDF ni una imagen, sino un payload estructurado con información equivalente a la salida de OCR, por ejemplo:

- campos extraídos del documento
- texto OCR bruto opcional
- confianza de extracción por campo opcional

- metadatos del documento

La prueba no busca evaluar OCR ni procesamiento documental complejo, sino la capacidad de construir un sistema que trabaje a partir de datos ya extraídos y sea capaz de resolver incertidumbre de negocio usando contexto y herramientas.

Semilla de datos proporcionada

Se proporcionará una semilla de datos compuesta por tres bloques:

1. Nuevas entradas post-OCR

Facturas nuevas sin resolver, equivalentes a las entradas que el sistema recibiría por API o evento.

Podrán incluir campos como:

- `document_id`
- `supplier_name`
- `supplier_tax_id`
- `invoice_number`
- `invoice_date`
- `currency`
- `base_amount`
- `tax_amount`
- `total_amount`
- `description`
- `country`
- `raw_ocr_text` opcional
- `field_confidence` opcional

2. Histórico resuelto

Facturas ya resueltas previamente, que actuarán como memoria o contexto del sistema.

Podrán incluir campos como:

- `supplier_name_raw`
- `supplier_tax_id`
- `canonical_supplier`
- `predicted_spend_category`
- `predicted_business_unit`
- `review_decision`
- `confidence`
- `resolved_at`

3. Datos de referencia

Opcionalmente, se proporcionarán también datos de referencia para facilitar la resolución, por ejemplo:

- proveedores canónicos conocidos
- catálogo de categorías
- catálogo de business units o cost centers
- reglas simples de revisión
- reglas básicas por país

Se espera que el sistema use el histórico y/o los datos de referencia como herramientas internas para ayudar a resolver las nuevas facturas.

Problema a resolver

Cada payload post-OCR representa una factura nueva todavía no resuelta.

A partir de esa entrada, la solución debe:

1. validar y normalizar los datos recibidos

2. almacenarlos en una base de datos operacional

3. ejecutar un proceso de resolución asistido por un agente basado en LLM

4. permitir que el agente consulte al menos una herramienta interna del sistema

5. devolver y persistir un resultado estructurado

6. permitir exportar incrementalmente los registros resueltos a una segunda capa analítica

Qué debe resolver el agente

Para esta prueba, no se requiere conocimiento profundo de contabilidad.

El componente agentic debe ayudar a resolver campos de negocio simplificados, definidos de forma cerrada y comprensible para cualquier perfil técnico.

Como mínimo, el agente debe inferir los siguientes campos:

- `canonical_supplier`
- `predicted_spend_category`
- `predicted_business_unit` o `predicted_cost_center`
- `review_decision`
- `confidence`
- `decision_explanation`

Definición de los campos a inferir

`canonical_supplier`

Nombre o identificador canónico del proveedor.

Objetivo:

- unificar variantes del mismo proveedor
- detectar coincidencias aproximadas con proveedores ya conocidos
- decidir si se trata de un proveedor nuevo o ya existente

Ejemplo:

- `Orange Espagne SA`
- `Orange Espagne, S.A.`
- `ORANGE ES`

Resultado esperado:

- `canonical_supplier = ORANGE ESPAGNE SA`

`predicted_spend_category`

Categoría de gasto sugerida.

Para esta prueba, la categoría debe elegirse a partir de un conjunto cerrado proporcionado en la semilla. Por ejemplo:

- `telecom`
- `software`
- `travel`
- `office_supplies`
- `professional_services`
- `utilities`
- `other`

Objetivo:

- usar proveedor, descripción e histórico para sugerir una categoría coherente

`predicted_business_unit` o `predicted_cost_center`

Área o unidad a la que parece corresponder el gasto.

No hace falta conocimiento contable. Puede tratarse como una clasificación simple a partir de un conjunto cerrado, por ejemplo:

- `operations`
- `sales`
- `marketing`
- `it`
- `general_admin`

Objetivo:

- usar proveedor, descripción e histórico para inferir qué área consume ese gasto

`review_decision`

Decisión sobre si el caso puede resolverse automáticamente o debe revisarse manualmente.

Valores esperados:

- `auto_approve`
- `needs_review`

Objetivo:

- reflejar incertidumbre real
- evitar resolver con demasiada seguridad casos ambiguos o con baja calidad de entrada

Ejemplos de casos que podrían requerir revisión:

- proveedor no visto antes
- tax ID ausente o inconsistente
- descripción insuficiente
- conflicto con el histórico
- baja confianza OCR en campos clave

`confidence`

Indicador de confianza de la resolución.

Puede implementarse como:

- valor numérico entre 0 y 1
- valor entero en una escala definida
- valor categórico si está bien justificado

No se exige una fórmula perfecta, pero sí una lógica razonable y documentada.

`decision_explanation`

Explicación breve y legible de por qué se tomó la decisión.

Ejemplo:

Se resolvió el proveedor como ORANGE ESPAGNE SA porque coincide con variantes históricas del nombre y el país. Las últimas facturas similares se clasificaron como telecom y se asignaron al área IT.

Qué no debe hacer el agente

No se espera que el agente resuelva tareas puramente deterministas o de limpieza técnica, como:

- parsear fechas
- convertir strings a números
- limpiar campos vacíos
- validar JSON
- comprobar operaciones aritméticas básicas

Estas tareas deben implementarse en la capa backend o de normalización.

El valor del agente debe estar en resolver incertidumbre de negocio usando contexto.

Uso esperado de herramientas

El agente no puede limitarse a un único prompt sin contexto.

Debe consultar al menos una herramienta interna del sistema. Por ejemplo:

- histórico de facturas resueltas
- tabla de proveedores canónicos
- catálogo de categorías
- catálogo de business units
- reglas simples de revisión
- reglas básicas por país

No es obligatorio usar un framework avanzado de agentes. Se puede implementar con:

- tool calling
- function calling
- router + funciones
- un bucle sencillo de decisión
- cualquier otra aproximación clara y justificable

Lo importante es que el uso de herramientas sea real y observable.

Requisitos funcionales

1. Ingesta

La solución debe permitir recibir nuevas entradas post-OCR por alguno de estos mecanismos:

- vía API
- o mediante un mecanismo equivalente basado en eventos

La opción más simple y recomendada es una API HTTP.

Como mínimo, debe ser posible:

- crear una entrada individual
- importar varias entradas en lote en formato JSON

Soporte CSV es opcional.

2. Normalización

Debe existir una capa explícita de normalización que trate, al menos, algunos de estos aspectos:

- normalización de nombres de proveedor
- fechas en formato consistente
- importes numéricos correctos
- monedas
- campos vacíos o nulos
- limpieza básica de la descripción

3. Persistencia operacional

Las facturas deben almacenarse en una base de datos operacional junto con, como mínimo:

- payload original
- payload normalizado
- estado de procesamiento
- resultado de resolución
- timestamps
- estado de exportación

4. Resolución agentic

Debe existir un componente basado en LLM que:

- resuelva los campos definidos en esta prueba
- use al menos una herramienta interna
- devuelva una salida estructurada
- incluya explicación breve
- incluya indicador de confianza
- decida si el caso requiere revisión manual

5. API de consulta

Debe poder consultarse por API:

- una factura
- su estado
- su resultado de resolución

6. Exportación incremental

Debe existir un mecanismo para exportar de forma incremental las facturas resueltas hacia una segunda capa de datos orientada a analítica.

Esa segunda capa puede implementarse en:

- PostgreSQL
- DuckDB
- archivos Parquet
- SQLite
- o equivalente

Endpoints mínimos esperados

La solución debe incluir endpoints equivalentes a estos:

- `POST /invoices`
- `POST /invoices/import`
- `GET /invoices/{id}`
- `POST /invoices/{id}/resolve`

- `POST /exports/run`

Se puede usar otra convención de rutas, siempre que la API sea clara y consistente.

Salida esperada del agente

La resolución debe persistirse y devolverse en formato estructurado. Por ejemplo:

```
{
  "canonical_supplier": "ORANGE ESPAGNE SA",
  "predicted_spend_category": "telecom",
  "predicted_business_unit": "it",
  "review_decision": "auto_approve",
  "confidence": 0.89,
  "decision_explanation": "El proveedor coincide con variantes históricas del nombre y país.
Las últimas facturas similares se clasificaron como telecom y se asignaron a IT."
}
```

Qué se espera de la solución

Se espera una solución que demuestre criterio técnico en:

- diseño de APIs
- normalización de datos
- modelado de persistencia
- uso razonable de LLMs
- integración de herramientas para el agente
- trazabilidad del proceso
- separación entre capa operacional y capa analítica

No se busca una solución perfecta ni de producción, sino una implementación clara, coherente y bien explicada.

Entregables obligatorios

El candidato debe entregar obligatoriamente:

1. Código fuente completo

Repositorio o archivo comprimido con el código necesario para ejecutar la solución.

2. README

Debe incluir como mínimo:

- descripción breve de la solución
- decisiones de diseño principales
- instrucciones de ejecución
- configuración de variables de entorno
- cómo probar la API
- limitaciones conocidas
- posibles mejoras futuras

3. API funcional

La API debe estar implementada y poder ejecutarse localmente.

4. Persistencia operacional

Debe existir una base de datos o almacenamiento equivalente donde se guarden:

- input original
- datos normalizados
- estado
- resultado de resolución
- estado de exportación

5. Componente de resolución agentic

Debe estar implementado e integrado en el flujo principal.

6. Al menos una herramienta real usada por el agente

Debe quedar claro qué herramienta usa el agente y cómo la consulta.

7. Exportación incremental

Debe existir un job, comando o endpoint que exporte registros resueltos a una segunda capa analítica.

8. Datos de ejemplo

La solución debe incluir un conjunto mínimo de datos de prueba o instrucciones para utilizar la semilla proporcionada.

Bonus opcionales

Se valorará positivamente, pero no es obligatorio, implementar alguno de los siguientes puntos.

Bonus de backend

- soporte CSV además de JSON
- validación con schemas
- documentación OpenAPI / Swagger
- tests automatizados
- idempotencia
- deduplicación
- logs estructurados
- manejo más robusto de errores

Bonus de agentes / LLM

- trazas de tool calls
- uso de function calling o structured output
- score de confianza mejor definido

- política explícita de fallback
- re-resolución de facturas
- separación clara entre razonamiento, tools y respuesta final

Bonus de datos

- mejor separación entre raw / normalized / resolved
- export incremental robusto
- control de cambios o reexportaciones
- data quality checks
- modelado analítico más claro
- preparación de una tabla final útil para training o reporting

Bonus de producto / negocio

- endpoint de revisión manual
- posibilidad de corregir una resolución y reinyectarla al histórico
- explicación más legible para usuario no técnico
- reglas adicionales por país
- inferencias adicionales documentadas y bien justificadas

Criterios de evaluación

1. Diseño de datos y modelado

Se evaluará:

- claridad del modelo
- separación entre input, normalización y resolución
- manejo de estados
- trazabilidad mínima

2. Calidad del backend y la API

Se evaluará:

- claridad de endpoints
- validación
- manejo de errores
- consistencia del diseño
- facilidad de probar la solución

3. Integración LLM / agente

Se evaluará:

- uso razonable del modelo
- uso real de herramientas
- salida estructurada
- explicación y confianza

- criterio para revisión manual

4. Robustez del pipeline

Se evaluará:

- persistencia correcta
- export incremental
- separación entre operacional y analítico
- claridad del flujo de procesamiento

5. Comunicación técnica

Se evaluará:

- calidad del README
- claridad de decisiones
- trade-offs
- explicación de limitaciones y mejoras futuras

Restricciones

No es necesario implementar:

- frontend
- autenticación real
- OCR complejo
- procesamiento documental completo
- despliegue cloud
- colas distribuidas reales
- fine-tuning
- multi-agent orchestration avanzada

Tecnologías

El stack es libre.

Se puede usar, por ejemplo:

- Python o TypeScript
- FastAPI / Flask / Express / Nest
- MongoDB / PostgreSQL / SQLite
- OpenAI API o equivalente
- DuckDB / PostgreSQL / Parquet para la capa analítica

Lo importante es la calidad de la solución, no una tecnología concreta.

Entrega esperada

La entrega debe incluir:

- repositorio o archivo comprimido con el código

- README
- instrucciones para ejecución local
- ejemplos de uso o colección Postman / cURL
- dataset mínimo de prueba o uso de la semilla proporcionada

Resumen del objetivo

La prueba busca evaluar la capacidad de construir un sistema que:

- reciba datos post-OCR
- los normalice y persista
- use un agente con herramientas para resolver incertidumbre de negocio
- exponga resultados por API
- y exporte los datos resueltos a una capa analítica